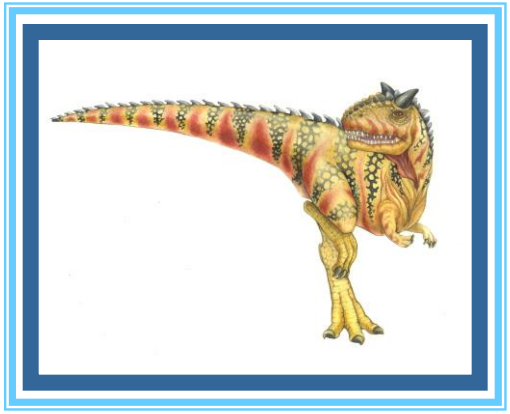
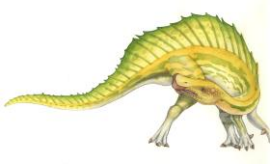
Chapter 3: Processes



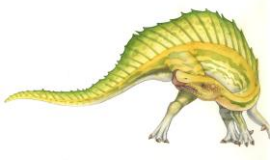
Outline

- Process Concept
- Process Scheduling
- Operations on Processes
- Interprocess Communication
- IPC in Shared-Memory Systems
- IPC in Message-Passing Systems



Objectives

- Identify the separate components of a process and illustrate how they are represented and scheduled in an operating system.
- Describe how processes are created and terminated in an operating system, including developing programs using the appropriate system calls that perform these operations.
- Describe and contrast interprocess communication using shared memory and message passing.

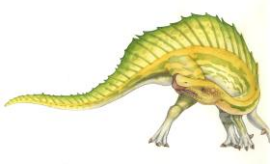


Process Concept

- An operating system executes a variety of programs that run as a process.
- **Process** – a program in execution; process execution must progress in sequential fashion. No parallel execution of instructions of a single process
- Multiple parts
  - The program code, also called **text section**
  - Current activity including **program counter**, processor registers
  - **Stack** containing temporary data
    - Function parameters, return addresses, local variables
  - **Data section** containing global variables
  - **Heap** containing memory dynamically allocated during run time

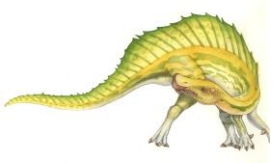
Process Concept (Cont.)

- Program is **passive** entity stored on disk (**executable file**); process is **active**
  - Program becomes process when an executable file is loaded into memory
- Execution of program started via GUI mouse clicks, command line entry of its name, etc.
- One program can be several processes
  - Consider multiple users executing the same program

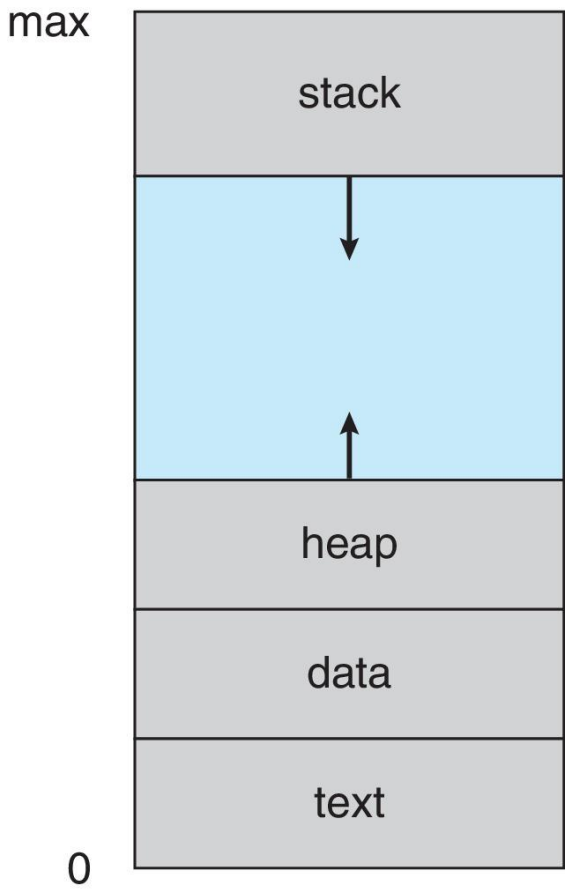


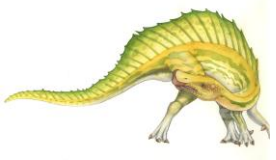
Controlul proceselor

- proces = abstractia rularii unui program (instr + date)
  - in fapt, este programul + starea executiei sale la un moment dat (reflectata de registrele CPU si valorile variabilelor din program)
- multiprogramarea/multitasking-ul (concurenta virtuala/logica sau fizica, pt multiprocesoare) impun nevoia unui model de proces
  - fiecare proces e tratat ca si cand ar avea propriul CPU (abstractia de masina virtuala/extinsa)
- managerul de procese
  - componenta kernel care gestioneaza procesele din sistem
  - foloseste tabela de procese (contine Process Control Blocks, PCB) si cozi de procese (gestionate de CPU)
  - planifica procesele pt executie pe CPU
- PCB-ul contine: Process ID (PID), prioritatea procesului, starea, valorile registrelor CPU, utilizarea memoriei, lista de fisiere deschise de proces, etc



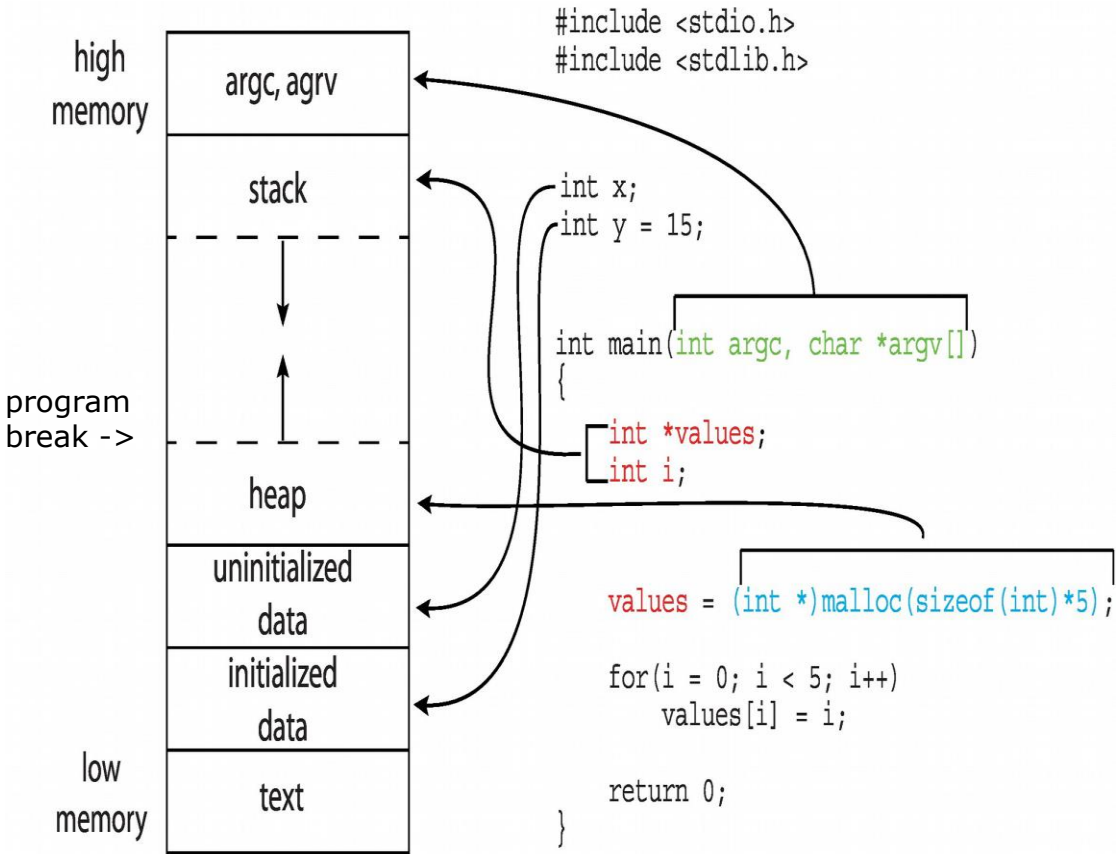
Process in Memory

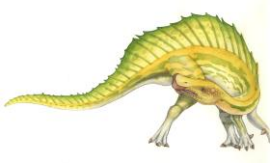




Imaginea unui program C in memorie

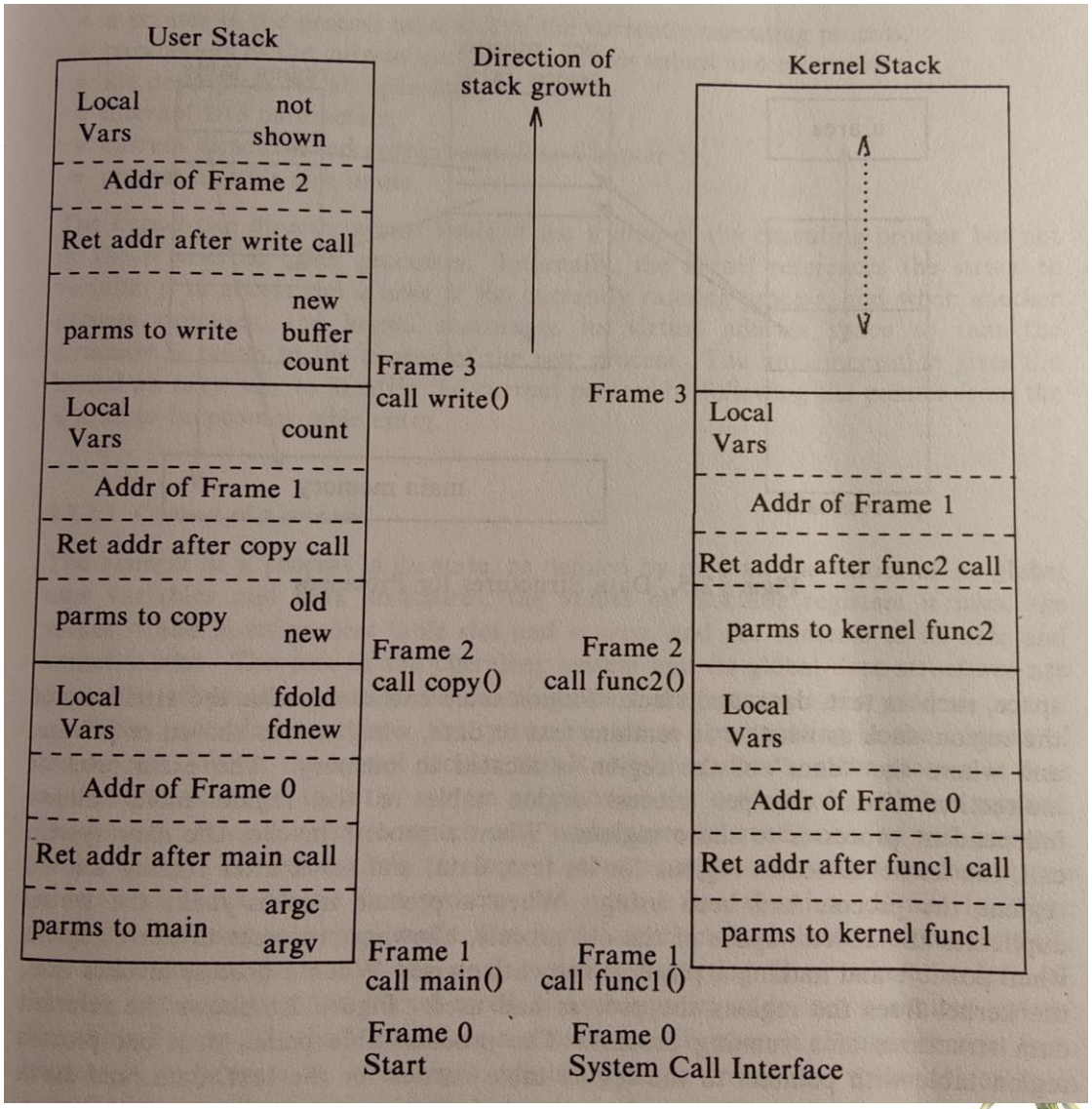
- codul si datele initializate sunt citite de *exec* din executabil (fisierul program de pe disc)
- datele neinitializate (*bss*) sunt initializate cu zero de *exec*
- heap-ul este gestionat ajutorul *program break*-ul (indica sfarsitul segmentului de date)
- valoarea break-ului este modificata prin apelurile de sistem *brk/sbrk*
  - cresterea valorii break-ului ⇔ alocare memorie dinamica
  - scaderea valorii break-ului ⇔ dealocare memorie dinamica
  - *sbrk(0)* intoarce valoarea curenta a program break-ului
  - in practica se evita folosirea *brk/sbrk* si se folosesc *malloc/free*
- stiva program (*call stack*) este gestionata cu ajutorul inregistrarilor de activare





Call stack

```
char buf[4096];

int main(int argc, char* argv[]) {
    int fdold, fdnew;

    if(argc != 3) return 0;

    if((fdold=open(argv[1],
            O_RDONLY)) < 0)
        exit(1);
   if((fdnew=creat(argv[2],0666))
            < 0)
        exit(1);

   copy(fdold, fdnew);
   exit(0);
}

void copy(int old, int new) {
   int count;

   while((count =read(old, buf,
            4096)) > 0)
        write(new, buf, count);
}
```



Process State

- As a process executes, it changes **state**
  - **New**: The process is being created
  - **Running**: Instructions are being executed
  - **Waiting**: The process is waiting for some event to occur
  - **Ready**: The process is waiting to be assigned to a processor
  - **Terminated**: The process has finished execution

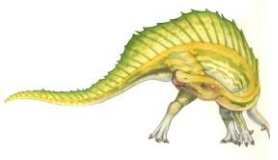
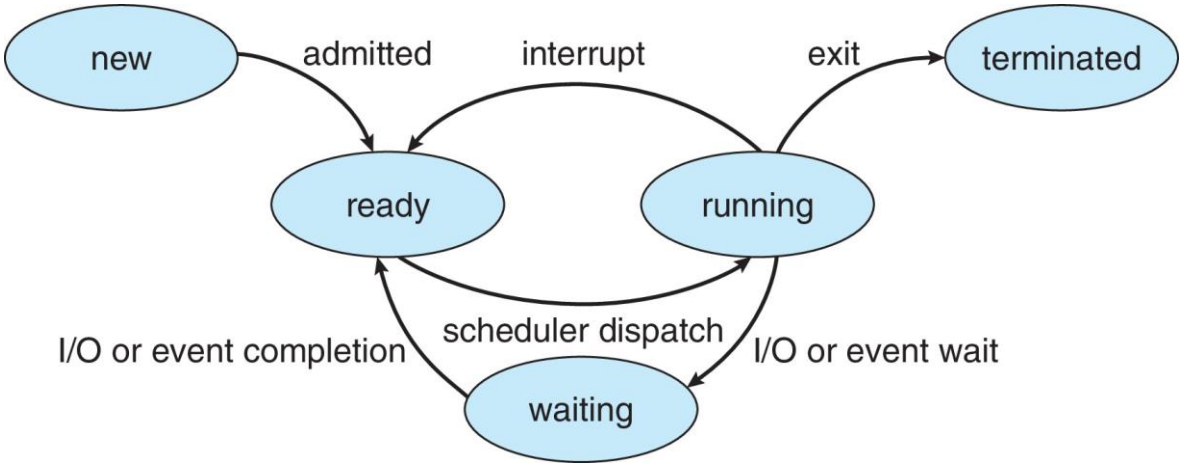
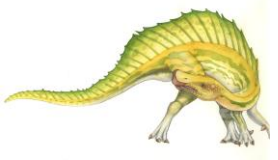
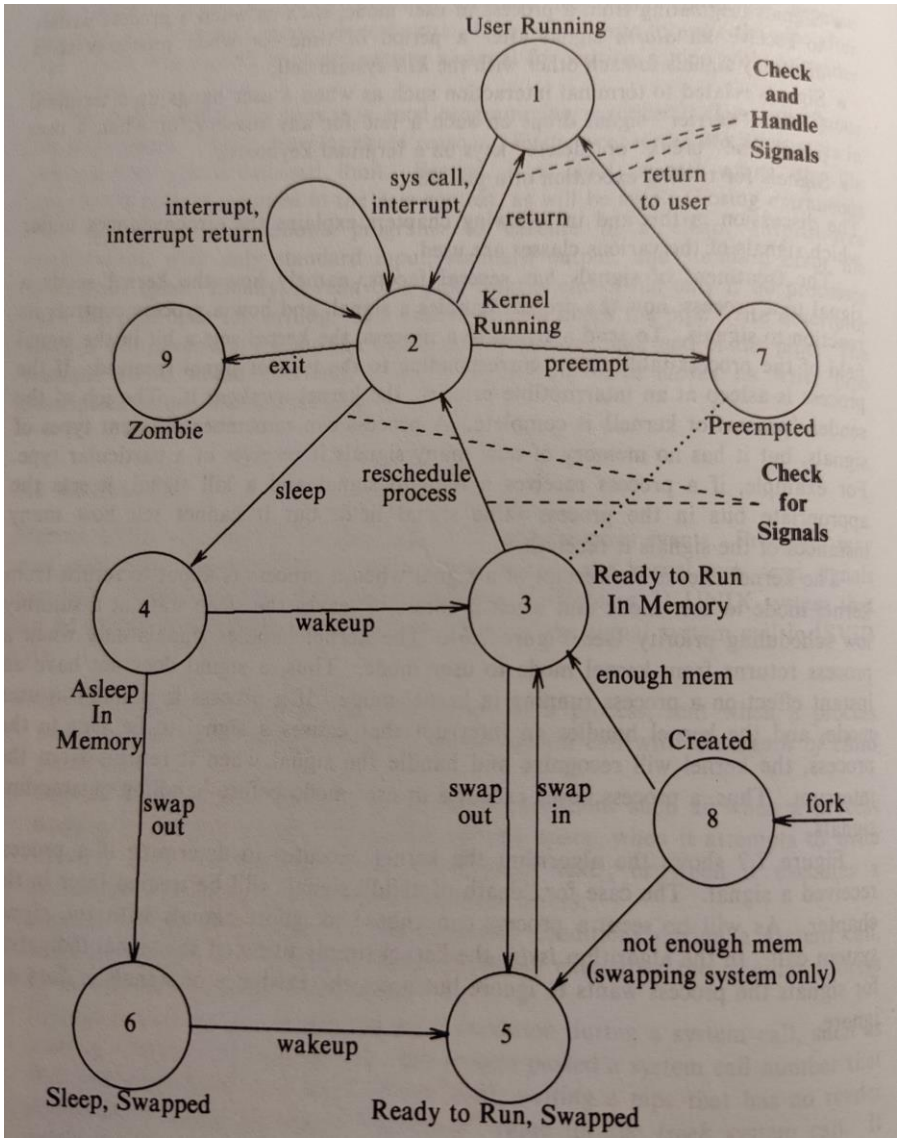


Diagram of Process State





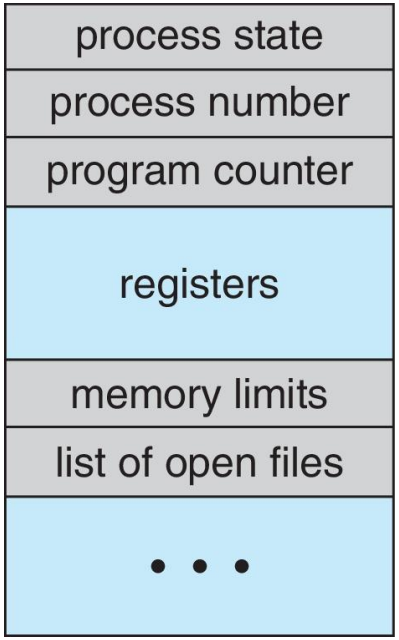
Process State Diagram – Unix example

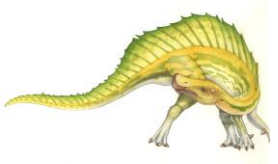


Process Control Block (PCB)

Information associated with each process(also called **task control block**)

- Process state – running, waiting, etc.
- Program counter – location of instruction to next execute
- CPU registers – contents of all process-centric registers
- CPU scheduling information- priorities, scheduling queue pointers
- Memory-management information – memory allocated to the process
- Accounting information – CPU used, clock time elapsed since start, time limits
- I/O status information – I/O devices allocated to process, list of open files

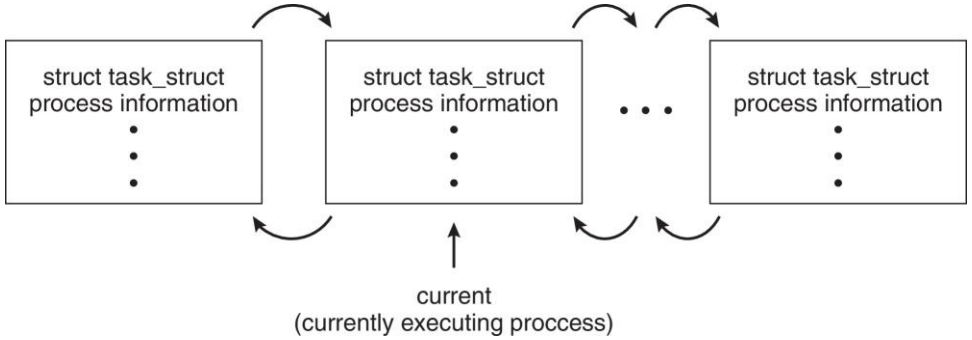


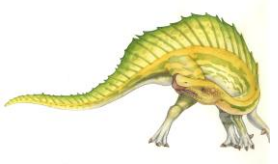


Process Representation in Linux

Represented by the C structure `task_struct`

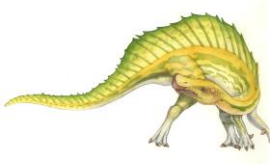
```
pid t_pid;                    /* process identifier */
long state;                   /* state of the process */
unsigned int time_slice       /* scheduling information */
struct task_struct *parent;/* this process's parent */
struct list_head children; /* this process's children */
struct files_struct *files;/* list of open files */
struct mm_struct *mm;         /* address space of this
process */
```



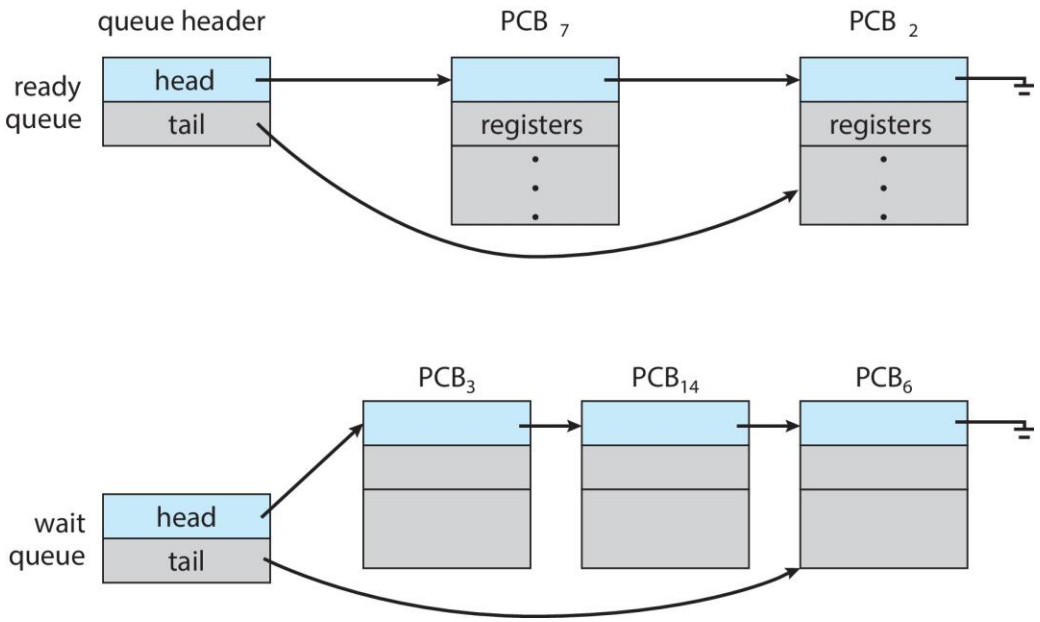


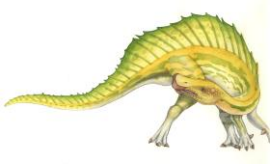
Process Scheduling

- **Process scheduler** selects among available processes for next execution on CPU core
- Goal -- Maximize CPU use, quickly switch processes onto CPU core
- Maintains **scheduling queues** of processes
  - **Ready queue** – set of all processes residing in main memory, ready and waiting to execute
  - **Wait queues** – set of processes waiting for an event (i.e., I/O)
  - Processes migrate among the various queues

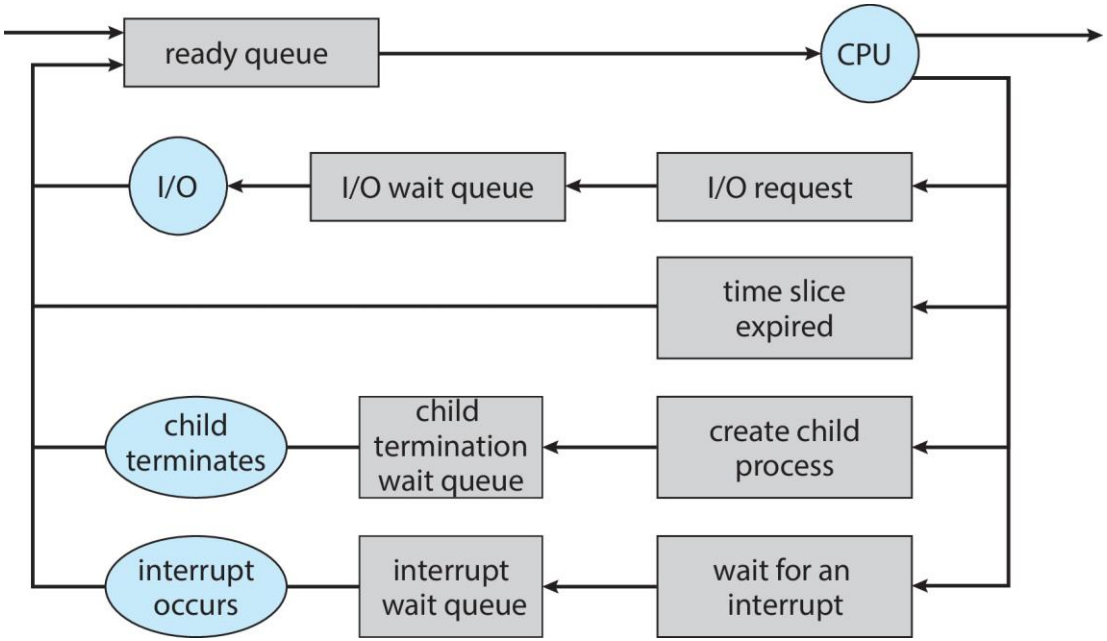


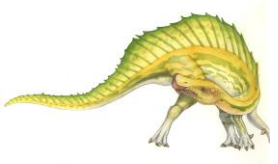
Ready and Wait Queues





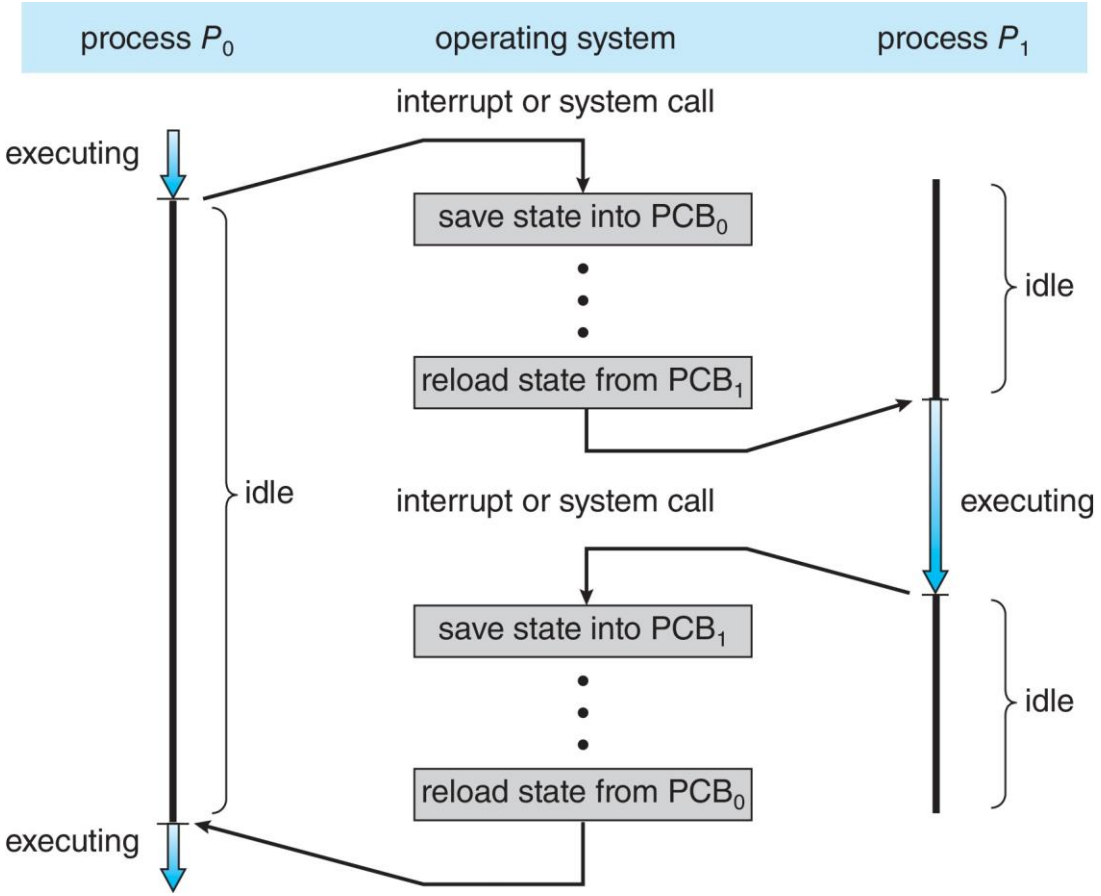
Representation of Process Scheduling





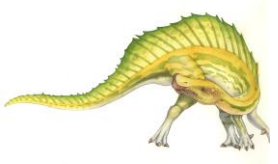
CPU Switch From Process to Process

A **context switch** occurs when the CPU switches from one process to another.



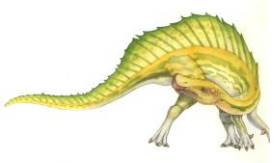
Context Switch

- When CPU switches to another process, the system must **save the state** of the old process and load the **saved state** for the new process via a **context switch**
- **Context** of a process represented in the PCB
- Context-switch time is pure overhead; the system does no useful work while switching
  - The more complex the OS and the PCB ➔ the longer the context switch
- Time dependent on hardware support
  - Some hardware provides multiple sets of registers per CPU ➔ multiple contexts loaded at once



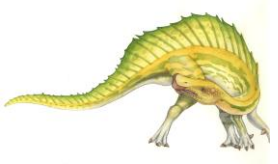
Gestiunea proceselor

- sistemul trebuie sa ofere mecanisme de creare si terminare a proceselor, ca si de asteptare a terminarii unui alt proces
- identificarea si gestiunea proceselor se face prin PID (*process ID*)
- *crearea proceselor*
  - model arborescent: un proces parinte creeaza procese copil, care la randul lor creeaza alte procese
  - la momentul crearii, procesul copil este o copie exacta a parintelui (cod + date)
  - implementarea duplicarii tine de optiuni si ratiuni de performanta, procesele pot partaja:
    - toate resursele
    - o parte dintre resurse
    - nu partajeaza resurse deloc



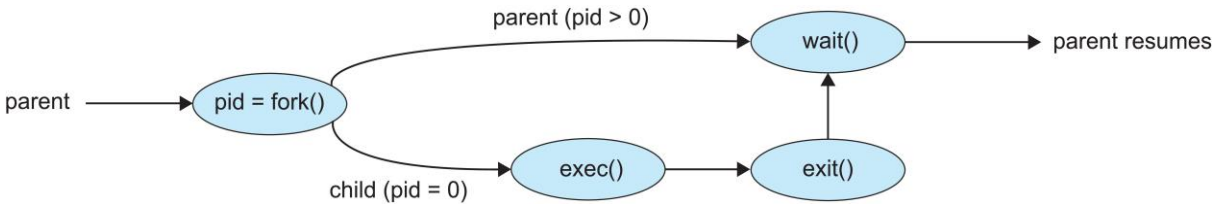
Gestiunea proceselor (cont.)

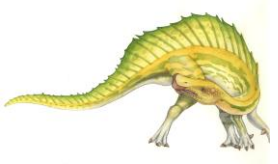
- crearea proceselor (cont.)
  - odata creat procesul copil, nu se poate face nici o presupunere asupra ordinii de executie a proceselor parinte si copil
    - ambele procese se executa concurent sub controlul planificatorului de procese din kernel
- terminarea proceselor
  - exista posibilitatea coordonarii explicite intre parinte si copil
    - *sincron*: procesul parinte asteapta terminarea copilului
    - asincron: procesul parinte nu asteapta terminarea copilului, dar este notificat de catre kernel la producerea evenimentului
  - daca parintele se termina inainte de copil, rolul de parinte este preluat de un alt proces (*init/systemd* in Unix/Linux)



Process Creation

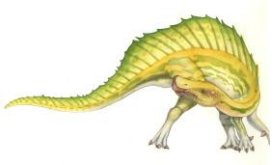
- Address space
  - Child duplicate of parent
  - Child has a program loaded into it
- UNIX examples
  - **`fork()`** system call creates new process
  - **`exec()`** system call used after a **`fork()`** to replace the process' memory space with a new program
  - Parent process calls **`wait()`** waiting for the child to terminate



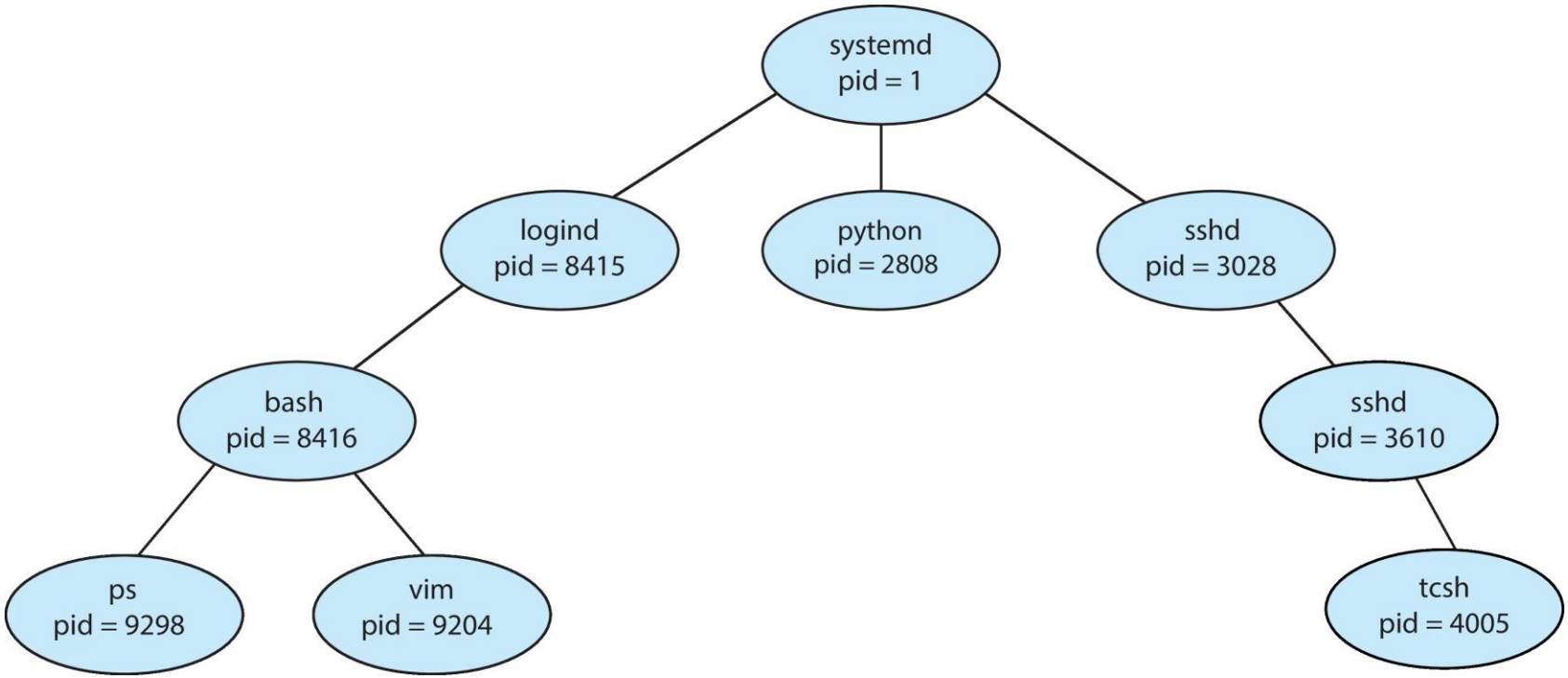


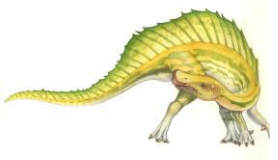
Executia fork in kernel

- kernelul verifica resursele disponibile
- aloca un PID unic si un PCB in tabela de procese
- verifica informatii de *quota* (de pilda daca utilizatorul nu are prea multe procese deschise)
- marcheaza starea procesului copil ca fiind *creat*
- copiaza PCB-ul parintelui in PCB-ul copilului (in particular, program counter-ul va fi util copilului pt reluarea executiei)
- incrementeaza referintele interne ale kernelului la directorul curent, directorul radacina si fisierele deschise
- duplica contextul de executie al parintelui (cod/text, date, stiva user si kernel)
- modifica in contextul procesului copil informatia necesara pt. ca acesta sa se poata recunoaste pe sine la reintoarcerea din apelul sistem
- parintele marcheaza starea copilului ca fiind *gata de rulare* si iese din kernel cu cod de retur egal cu noul PID alocat copilului
- copilul se intoarce si el din *fork* cu cod de retur zero



A Tree of Processes in Linux



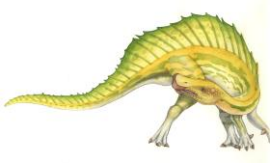


C Program Forking Separate Process

```
#include <sys/types.h>
#include <stdio.h>
#include <unistd.h>

int main()
{
pid_t pid;

    /* fork a child process */
    pid = fork();

    if (pid < 0) { /* error occurred */
       fprintf(stderr, "Fork Failed");
       return 1;
    }
    else if (pid == 0) { /* child process */
       execlp("/bin/ls","ls",NULL);
    }
    else { /* parent process */
       /* parent will wait for the child to complete */
       wait(NULL);
       printf("Child Complete");
    }

    return 0;
}
```

Terminarea proceselor

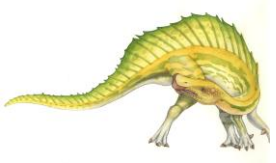
- *normal*, prin terminarea programului (“iesirea” din functia *main* in C)
- *anormal*, ca urmare a unor exceptii software (primirea unor semnale)
- dpdv tehnic, prin apel de sistem *exit*
  - intoarce starea procesului care se termina
    - recuperata de parinte cu ajutorul apelului sistem *wait*
  - dealoca resursele procesului
  - permite tratarea specifica a evenimentului prin handlere asociate terminarii procesului cu ajutorul *atexit*
    - de ex: tratarea specifica a dealocarii memoriei, a inchiderii fisierelor deschise, etc



Terminarea proceselor (cont.)

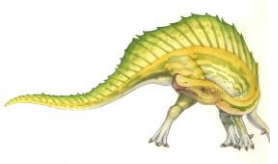
- procesul parinte poate astepta terminarea copilului
  - *sincron*, cu ajutorul apelului blocant *wait*
  - *asincron*: apeleaza *wait* (neblocant) doar cand e notificat de kernel ca procesul copil s-a terminat (semnal *SIGCHLD*)
  - in ambele cazuri, *wait* furnizeaza parintelui informatia de stare a copilului

    pid = wait(&status);
- parintele nu asteapta terminarea copilului (nu cheama *wait*)
  - procesul copil intra in starea de *zombie*
    - fara PCB, kernelul mentine doar informatie minimala, i.e. cod de iesire, PID, informatie “contabila” despre procesul copil
- parintele nu apeleaza *wait* si termina inaintea copilului
  - procesul copil devine *orfan*
  - in Unix, devine copilul procesului *init*, primul proces pornit de sistem (PID = 1)
  - *init* va executa un *wait* (neblocant) cand primeste *SIGCHLD* de la kernel la terminarea copilului



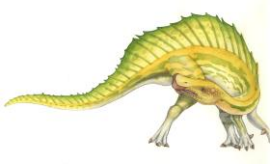
Process Termination

- Parent may terminate the execution of children processes using the **`abort()`** system call. Some reasons for doing so:
  - Child has exceeded allocated resources
  - Task assigned to child is no longer required
  - The parent is exiting, and the operating systems does not allow a child to continue if its parent terminates
- Some operating systems do not allow child to exists if its parent has terminated. If a process terminates, then all its children must also be terminated.
  - **cascading termination.** All children, grandchildren, etc., are terminated.
  - The termination is initiated by the operating system.



Interprocess Communication

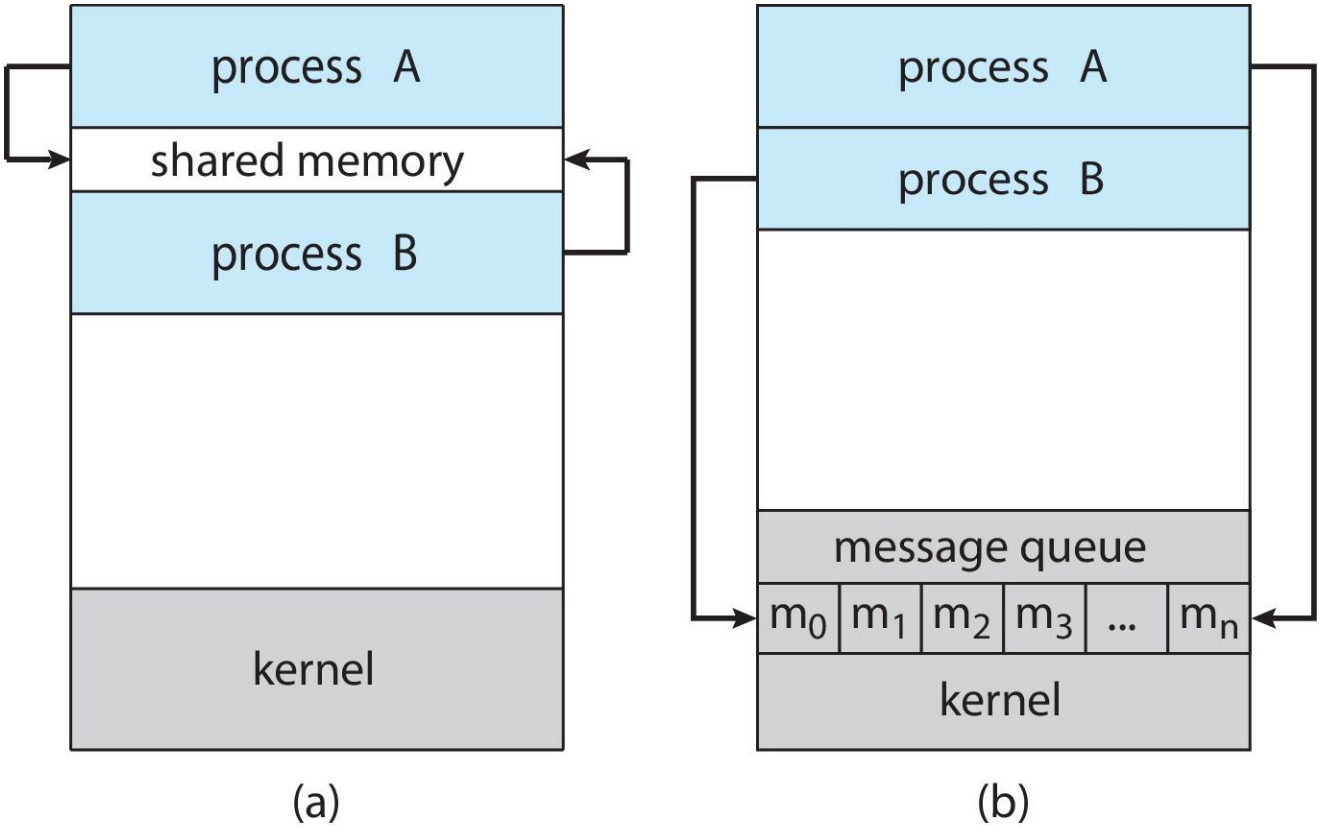
- Processes within a system may be ***independent*** or ***cooperating***
- Cooperating process can affect or be affected by other processes, including sharing data
- Reasons for cooperating processes:
  - Information sharing
  - Computation speedup
  - Modularity
  - Convenience
- Cooperating processes need **interprocess communication** (**IPC**)
- Two models of IPC
  - **Shared memory**
  - **Message passing**

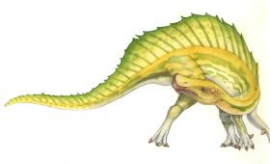


Communications Models

(a) Shared memory.

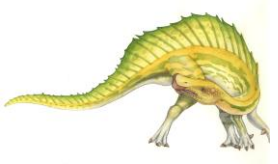
(b) Message passing.





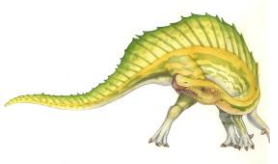
Producer-Consumer Problem

- Paradigm for cooperating processes:
  - *producer* process produces information that is consumed by a *consumer* process
- Two variations:
  - **unbounded-buffer** places no practical limit on the size of the buffer:
    - Producer never waits
    - Consumer waits if there is no buffer to consume
  - **bounded-buffer** assumes that there is a fixed buffer size
    - Producer must wait if all buffers are full
    - Consumer waits if there is no buffer to consume



IPC – Shared Memory

- An area of memory shared among the processes that wish to communicate
- The communication is under the control of the users processes not the operating system.
- Major issues is to provide mechanism that will allow the user processes to synchronize their actions when they access shared memory.
- Synchronization is discussed in great details in Chapters 6 & 7.

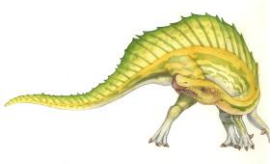


Bounded-Buffer – Shared-Memory Solution

- Shared data

```
#define BUFFER_SIZE 10
typedef struct {
  . . .
} item;

item buffer[BUFFER_SIZE];
int in = 0;
int out = 0;
```

- Solution is correct, but can only use **`BUFFER_SIZE-1`** elements

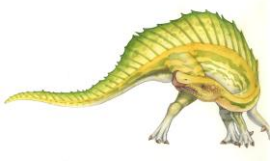


Producer Process – Shared Memory

```
item next_produced;

while (true) {
   /* produce an item in next produced */
   while (((in + 1) % BUFFER_SIZE) == out)
        ; /* do nothing */
   buffer[in] = next_produced;
   in = (in + 1) % BUFFER_SIZE;
}
```

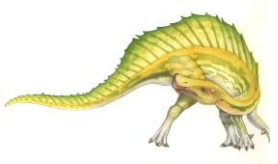
Consumer Process – Shared Memory

```
item next_consumed;

while (true) {
        while (in == out)
                ; /* do nothing */
        next_consumed = buffer[out];
        out = (out + 1) % BUFFER_SIZE;

        /* consume the item in next consumed */
}
```



What about Filling all the Buffers?

- Suppose that we wanted to provide a solution to the consumer-producer problem that fills **all** the buffers.
- We can do so by having an integer **`counter`** that keeps track of the number of full buffers.
- Initially, **`counter`** is set to 0.
- The integer **`counter`** is incremented by the producer after it produces a new buffer.
- The integer **`counter`** is and is decremented by the consumer after it consumes a buffer.

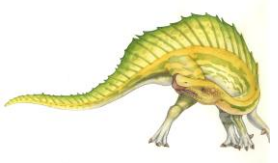


Producer

```
while (true) {
        /* produce an item in next produced */

        while (counter == BUFFER_SIZE)
                ; /* do nothing */
        buffer[in] = next_produced;
        in = (in + 1) % BUFFER_SIZE;
        counter++;
}
```

Consumer

```
while (true) {
        while (counter == 0)
                ; /* do nothing */
        next_consumed = buffer[out];
        out = (out + 1) % BUFFER_SIZE;
         counter--;
        /* consume the item in next consumed */
}
```



Race Condition

- **`counter++`** could be implemented as

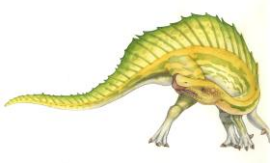
```
register1 = counter
register1 = register1 + 1
counter = register1
```

- **`counter--`** could be implemented as

```
register2 = counter
register2 = register2 - 1
counter = register2
```

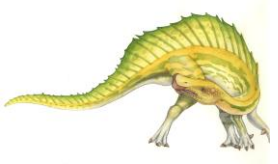
- Consider this execution interleaving with "count = 5" initially:

S0: producer execute `register1 = counter` {register1 = 5}
S1: producer execute `register1 = register1 + 1` {register1 = 6}
S2: consumer execute `register2 = counter` {register2 = 5}
S3: consumer execute `register2 = register2 - 1` {register2 = 4}
S4: producer execute `counter = register1` {counter = 6 }
S5: consumer execute `counter = register2` {counter = 4}



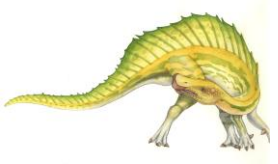
IPC – Message Passing

- Processes communicate with each other without resorting to shared variables
- IPC facility provides two operations:
  - **`send`**(*message*)
  - **`receive`**(*message*)
- The *message* size is either fixed or variable



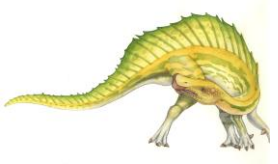
Message Passing (Cont.)

- If processes *P* and *Q* wish to communicate, they need to:
  - Establish a ***communication link*** between them
  - Exchange messages via send/receive
- Implementation issues:
  - How are links established?
  - Can a link be associated with more than two processes?
  - How many links can there be between every pair of communicating processes?
  - What is the capacity of a link?
  - Is the size of a message that the link can accommodate fixed or variable?
  - Is a link unidirectional or bi-directional?



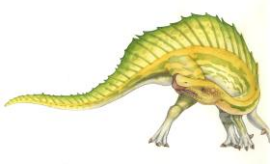
Implementation of Communication Link

- Physical:
  - Shared memory
  - Hardware bus
  - Network
- Logical:
  - Direct or indirect
  - Synchronous or asynchronous
  - Automatic or explicit buffering



Direct Communication

- Processes must name each other explicitly:
  - **`send`** (*P, message*) – send a message to process P
  - **`receive`**(*Q, message*) – receive a message from process Q
- Properties of communication link
  - Links are established automatically
  - A link is associated with exactly one pair of communicating processes
  - Between each pair there exists exactly one link
  - The link may be unidirectional, but is usually bi-directional

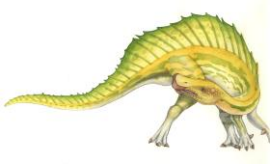


Indirect Communication

- Messages are directed and received from mailboxes (also referred to as ports)
  - Each mailbox has a unique id
  - Processes can communicate only if they share a mailbox
- Properties of communication link
  - Link established only if processes share a common mailbox
  - A link may be associated with many processes
  - Each pair of processes may share several communication links
  - Link may be unidirectional or bi-directional

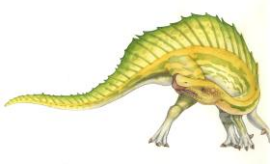
Indirect Communication (Cont.)

- Operations
  - Create a new mailbox (port)
  - Send and receive messages through mailbox
  - Delete a mailbox
- Primitives are defined as:
  - **`send`**(*A, message*) – send a message to mailbox A
  - **`receive`**(*A, message*) – receive a message from mailbox A



Indirect Communication (Cont.)

- Mailbox sharing
  - P_1, P_2, and P_3 share mailbox A
  - P_1, sends; P_2 and P_3 receive
  - Who gets the message?
- Solutions
  - Allow a link to be associated with at most two processes
  - Allow only one process at a time to execute a receive operation
  - Allow the system to select arbitrarily the receiver. Sender is notified who the receiver was.



Synchronization

Message passing may be either blocking or non-blocking

- **Blocking** is considered **synchronous**
  - **Blocking send** -- the sender is blocked until the message is received
  - **Blocking receive** -- the receiver is blocked until a message is available
- **Non-blocking** is considered **asynchronous**
  - **Non-blocking send** -- the sender sends the message and continue
  - **Non-blocking receive** -- the receiver receives:
    - A valid message, or
    - Null message
- Different combinations possible
  - If both send and receive are blocking, we have a **rendezvous**